Unit-III
Virtualization and Containers

Content

- Introduction,
- Characteristics of Virtualized Environments,
- Taxonomy of Virtualization Techniques,
- Virtualization and Cloud Computing,
- Pros and Cons of Virtualization,
- Technology Examples,
- Contemporary Virtualization Techniques,
- Containers,
- Container Orchestration,
- Docker and Kubernetes.

Content

- Virtualization 3.1
  - Introduction 3.1
  - Characteristics of Virtualized Environments 3.3
  - Taxonomy of Virtualization Techniques 3.6
    - Execution Virtualization 3.7
    - Other Types of Virtualization 3.16
  - Virtualization and Cloud Computing 3.17
  - Pros and Cons of Virtualization 3.19
  - Technology Examples 3.21
    - Xen: Paravirtualization 3.21
    - VMware: Full Virtualization 3.22
    - Microsoft Hyper-V 3.27
  - Contemporary Virtualization Techniques
- Containers,
  - Container Orchestration,
    - What is Containers ?
    - Primary Terminologies
    - Containers vs. Virtual Machines (VMs)
    - What is Containerization?
    - Step-by-Step Process: How Containers Work?
    - Benefits of Containers
  - Docker and Kubernetes.

Introduction

- Virtualization technology is one of the **fundamental components** of Cloud computing, especially in case of **infrastructure-based services**.
- It allows creation of **secure, customizable, and isolated execution environment** for running applications, even if they are untrusted, without affecting other user's applications.
- Virtualization is **a large umbrella of technologies and concepts** that are meant to provide an **abstract environment**—whether this is virtual hardware or operating system—to run applications.

Introduction

- This term is often synonymous with ***hardware virtualization***, which plays fundamental role in efficiently delivering ***Infrastructure-as-a-Service*** solutions for Cloud computing.
- virtualization technologies have a **long trail** in the history of computer science and have **come into many flavors** by providing virtual environments at **operating system level, programming language level, and application level**.
- Virtualization technologies not only provide a virtual environment for executing applications, but also for storage, memory, and networking.

Virtualization: reasons for renewed interest

- Virtualization technologies have gained a renewed interested recently due to the confluence(merging.) of different phenomena
  - ***Increased performance and computing capacity:*** Almost all **modern PCs have resources enough to host a virtual machine manager and execute** a virtual machine with a by far acceptable performance.
  - ***Underutilized hardware and software resources:*** Hardware and software underutilization is occurring due to (1**) the increased performance and computing capacity, and (2) effect of limited or sporadic use of resources.** Using these resources for other purposes after hours could improve the efficiency of the IT infrastructure. In order to transparently provide such a service, it would be necessary to deploy a completely separate environment, which can be achieved through virtualization.
  - ***Lack of space:*** The **continuous need** for additional capacity, whether this is **storage or compute power, makes data centers grow quickly**. This condition along with hardware underutilization led to the diffusion of a technique called server consolidation, for which virtualization technologies are fundamental.

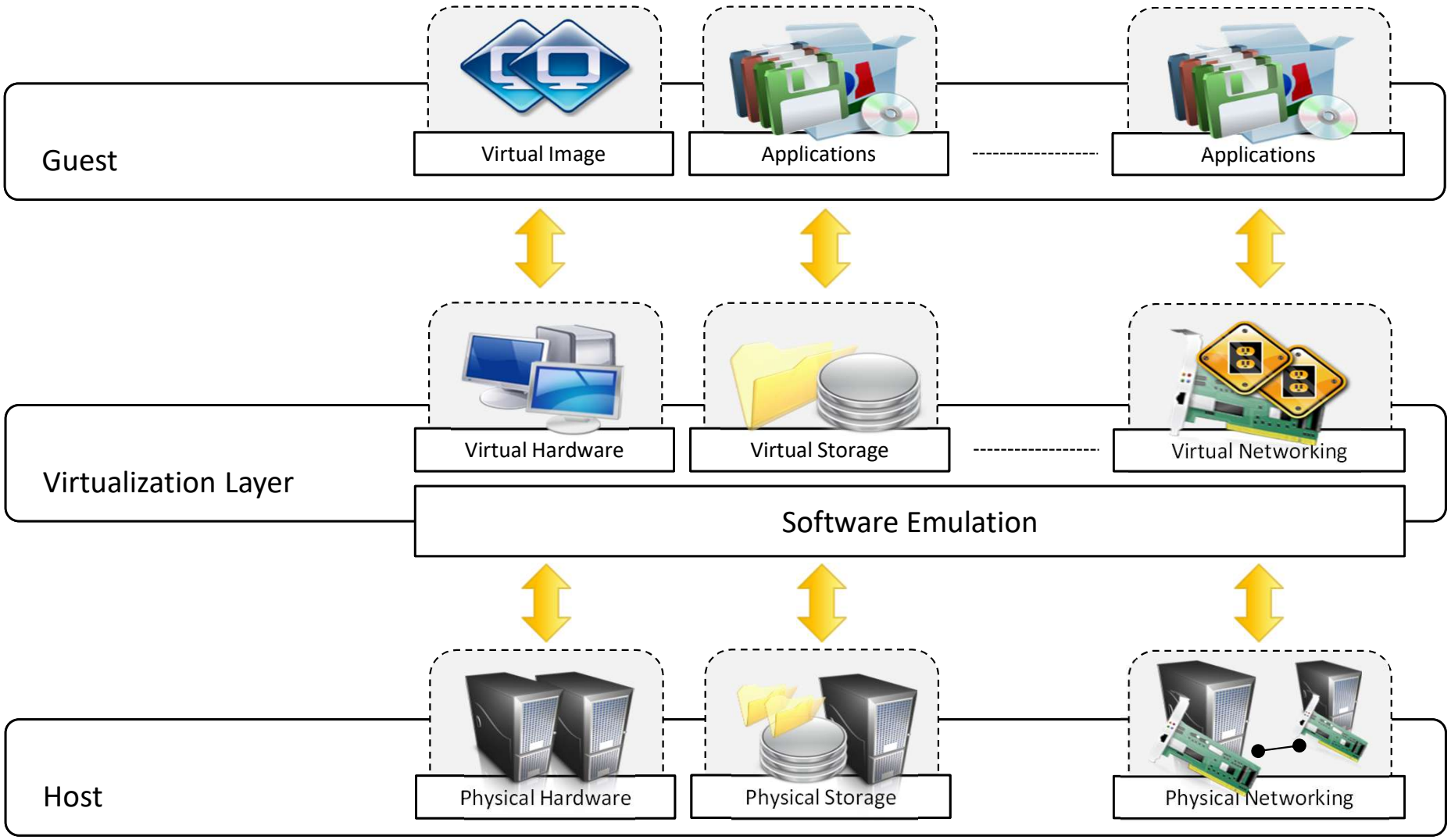
Virtualization: reasons for renewed interest

- Contd..
  - ***Greening initiatives:*** Recently, companies are increasingly looking for ways to **reduce the amount of energy** they consume and to **reduce their carbon footprint**. Hence, reducing the number of servers through server consolidation will definitely **reduce the impact of cooling and power consumption** of a data center. Virtualization technologies can provide an efficient way of consolidating servers.
  - ***Rise of administrative costs:*** **Power consumption and cooling costs have now become higher** than the cost of the **IT equipment.** Virtualization can help in reducing the number of required servers for a given workload, thus reducing the cost of the administrative personnel.

Virtualization reference model

- Virtualization is a broad concept and it refers to the creation of a virtual version of something, whether this is hardware, software environment, storage, or network.
- In a virtualized environment there are **three major** components: ***guest*, *host*, and *virtualization layer*.**
- The ***guest*** represents the **system component** that interacts with the virtualization layer rather than with the host as it would normally happen.
- The ***host*** represents the **original environment** where the guest is supposed to be managed.
- The ***virtualization layer*** is responsible for **recreating the same or a different environment** where the guest will operate.

Virtualization reference model



Characteristics of virtualized environments

- *Increased Security:* The ability to control the execution of a guest in a completely transparent manner opens new possibilities for delivering a secure, controlled execution environment.
  - The virtual machine represents an **emulated environment** in which the guest is executed. All the operations of the guest are generally performed against the virtual machine, which then translates and applies them to the host.
  - This level of indirection allows the virtual machine manager to ***control* and *filter*** the activity of the guest, thus preventing some harmful operations from being performed.
  - **Resources exposed by the host** can then be hidden or simply protected from the guest. Moreover, **sensitive information** that is contained in the host can be naturally hidden without the need of installing complex security policies. Increased security is a requirement when dealing with untrusted code.

Characteristics of virtualized environments contd…

- ***Managed Execution:*** Virtualization of the execution environment does not only allow the increased security but a wider range of features can be implemented. In particular, ***sharing, aggregation, emulation*, and *isolation*** are the most relevant.

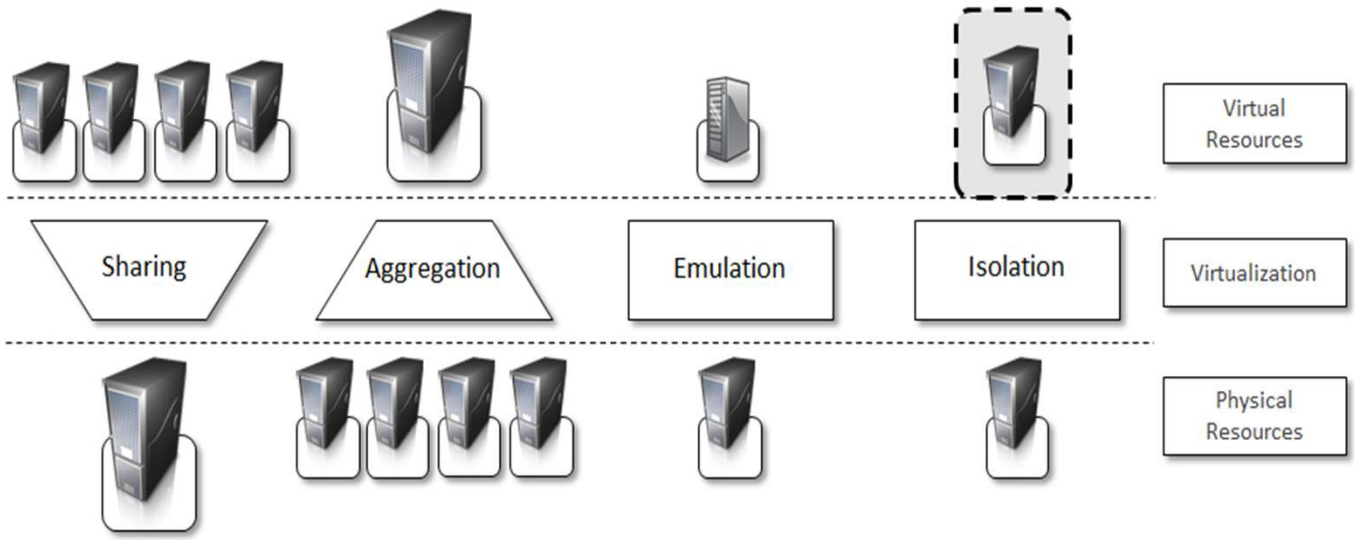


Fig- Functions Enabled by Managed Execution

Characteristics of virtualized environments contd…

- *Sharing:*
  - Virtualization allows the creation of a separate computing environment within the same host. In this way it is possible to fully exploit the capabilities of a powerful guest, which would be otherwise underutilized.
  - Sharing is a particularly important feature in virtualized data centers, where this basic feature is used to reduce the number of active servers and limit power consumption.
- *Aggregation.*
  - It is not only possible to share the physical resource among several guests, but virtualization also allows the aggregation, which is the opposite process. A group of separate hosts can be tied together and represented to guests as a single virtual host. This function is naturally implemented in middleware for distributed computing and a classical example is represented by cluster management software, which harnesses the physical resources of a homogeneous group of machines and represents them as a single resource.

Characteristics of virtualized environments contd…

- *Emulation.* Guests are executed within an environment that is controlled by the virtualization layer, which ultimately is a program. This allows for controlling and tuning the environment that is exposed to guests.
  - For instance, a complete different environment with respect to the host can be emulated, thus allowing the execution of guests requiring specific characteristics that are not present in the physical host.
  - hardware virtualization solutions are able to provide virtual hardware and emulate a particular kind of device such as *SCSI (Small Computer System Interface)* devices for file IO, without the hosting machine having such hardware installed.
  - Old and legacy software, which does not meet the requirements of current systems, can be run on emulated hardware without any need of changing their code. This is possible either emulating the required hardware architecture or within a specific operating system sandbox, such as the MS-DOS mode in Windows 95/98.

Characteristics of virtualized environments contd…

- *Isolation.* Virtualization allows providing guests—whether they are operating systems, applications, or other entities—with a complete separate environment, in which they are executed. The guest performs its activity by interacting with an abstraction layer, which provides access to the underlying resources.
  - Isolation brings several benefits, for example it allows multiple guests to run on the same host without each of them interfering with the other.
  - Secondly, it provides a separation between the host and the guest. The virtual machine can filter the activity of the guest and prevent harmful operations against the host.

Characteristics of virtualized environments contd…

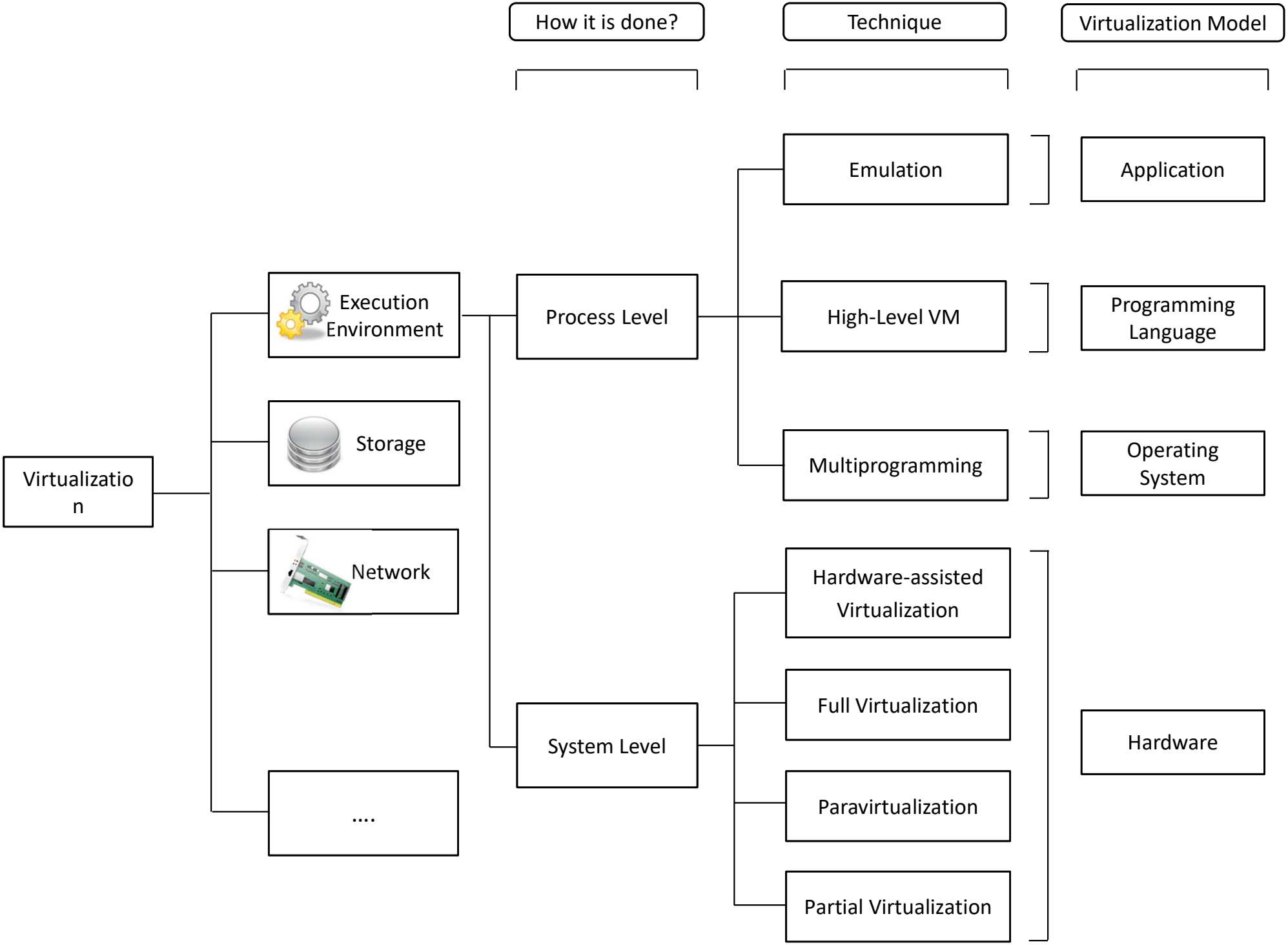
- *Performance tuning:* This feature is a reality at present time, given the considerable advances in hardware and software supporting virtualization. It becomes easier to control the performance of the guest by finely tuning the properties of the resources exposed through the virtual environment. This provides means to effectively implement a Quality of Service infrastructure that more easily fulfill the service level agreement established for the guest.
  - For instance software implementing hardware virtualization solutions can expose to a guest operating system only a fraction of the memory of the host machine or to set the maximum frequency of the processor of the virtual machine.
  - Another advantage of managed execution is that sometimes it allows easily capturing the state of the guest, persisting it, and resuming its execution.
    - This, for example, allows virtual machine managers such as *Xen Hypervisor* to stop the execution of a guest operating system, to move its virtual image into another machine, and to resume its execution in a completely transparent manner. This technique is called *virtual machine migration* and constitutes an important feature in virtualized data centers for optimizing their efficiency in serving applications demand.

Characteristics of virtualized environments contd…

- *Portability:* The concept of portability applies in different ways according to the specific type of virtualization considered.
  - In the case of a hardware virtualization solution the guest is packaged into a virtual image that, in most of the cases, can be safely moved and executed on top of different virtual machines.
  - In the case of programming level virtualization, as implemented by the JVM or the .NET runtime, the binary code representing application components (jars or assemblies), can be run without any recompilation on any implementation of the corresponding virtual machine.
  - This makes the application development cycle more flexible and application deployment very straightforward: one version of the application, in most of the cases, is able to run on different platforms with no changes.

Taxonomy of Virtualization Techniques

- Virtualization covers a wide range of emulation techniques that are applied to different areas of computing. A classification of these techniques helps to better understand their characteristics and use.
  - Virtualization is mainly used **to emulate *execution environments*, *storage*, and *networks*.**
  - Among these categories *execution virtualization* constitutes the oldest, **most popular, and most developed area.**
  - We can divide these execution virtualization techniques into two major categories by considering the type of host they require.
    - *Process level* techniques are implemented on top of an existing operating system, which has full control of the hardware.
    - *System level* techniques are implemented directly on hardware and do not require—or require a minimum support from—an existing operating system.


How it is done?
Technique
Virtualization Model
Virtualization
Execution Environment
Storage
Network
....
Process Level
System Level
Emulation
High-Level VM
Multiprogramming
Hardware-assisted Virtualization
Full Virtualization
Paravirtualization
Partial Virtualization
Application
Programming Language
Operating System
Hardware

Execution Virtualization

- Execution virtualization includes all those techniques whose aim is to **emulate an execution environment** that is separate from the one hosting the virtualization layer.
- All these techniques concentrate their interest on providing support for the execution of programs, whether these are the **operating system, a binary specification of a program compiled** against an abstract machine model, or an application.
- Therefore, execution virtualization can **be implemented directly on top of the hardware, by the operating system, an application, or libraries dynamically or statically** linked against an application image.

Machine Reference Model

- Virtualizing an execution environment at **different levels of the computing stack** requires a reference model that defines the interfaces between the levels of abstractions, which hide implementation details.
- From this perspective, virtualization techniques actually **replace one of the layers and intercept** the calls that are directed towards it.
- Therefore, a **clear separation between layers** simplifies their implementation, which only requires the emulation of the interfaces and a proper interaction with the underlying layer.
- Modern computing systems can be expressed in terms of the reference model described in the figure.

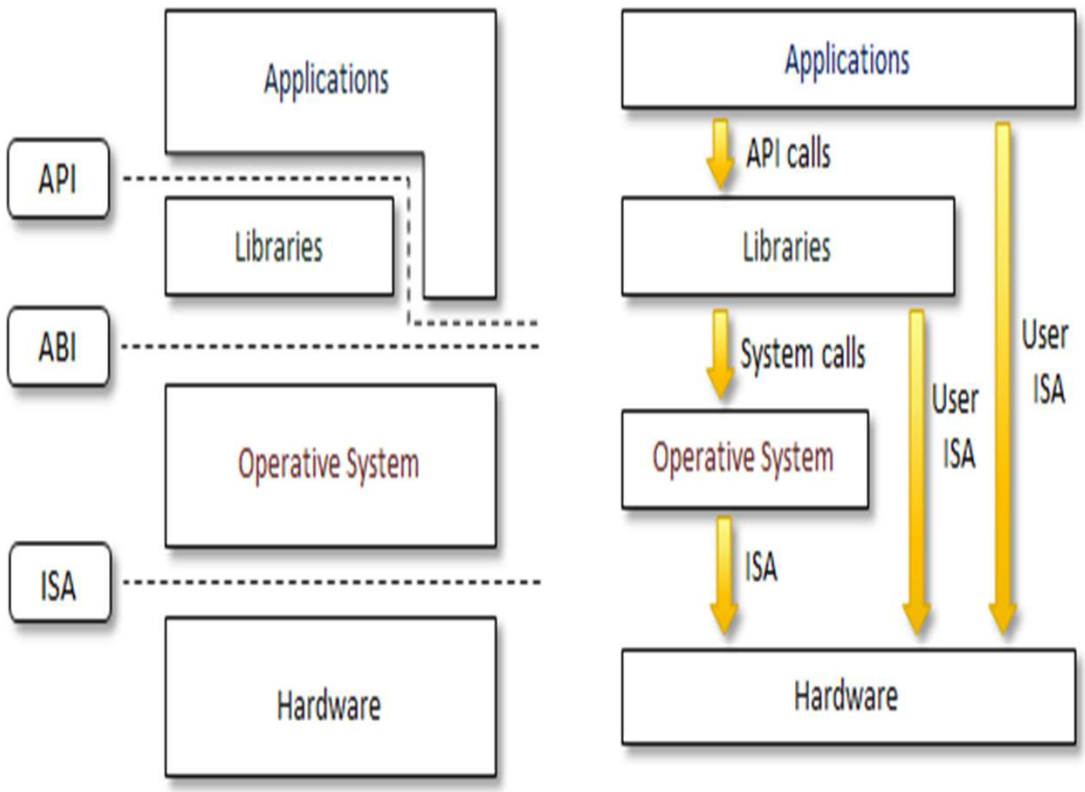


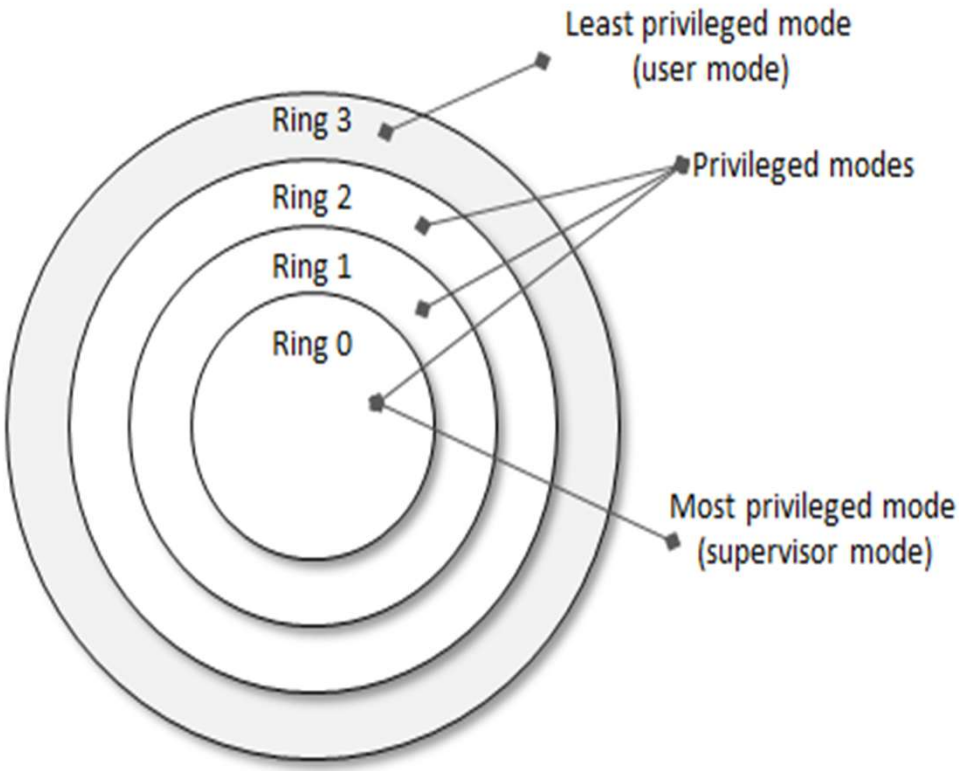
Fig: Machine reference Model

Machine Reference Model contd…

- At **the bottom layer**, the model for the hardware is expressed in terms of the ***Instruction Set Architecture (ISA)*****,** which defines the instruction set for the **processor, registers, memory, and interrupts management.**
- ISA is the **interface** between **hardware and software** and it is **important** for the OS developer (*System ISA*), and developers of applications that directly manage the underlying hardware (*User ISA*).
- The ***Application Binary Interface*** *(ABI)* separates the **operating system layer from the applications and libraries,** which are managed by the OS.
- ABI covers details such as **low-level data types, alignment, and call conventions** and defines a format for executable programs. **System calls are defined at this level**. This interface allows portability of applications and libraries across operating systems that implement the same ABI.
- The **highest level** of abstraction is represented by the ***Application Programming Interface (API)*****,** which interfaces applications to libraries and/or the underlying operating system.
- The machine level resources such as processor registers and main memory capacities are used to perform the operation in the hardware level of CPU.
- Such layered approach **simplifies the development and implementation of computing systems** and also simplifies the implementation of multi-tasking and the co-existence of multiple executing environments.

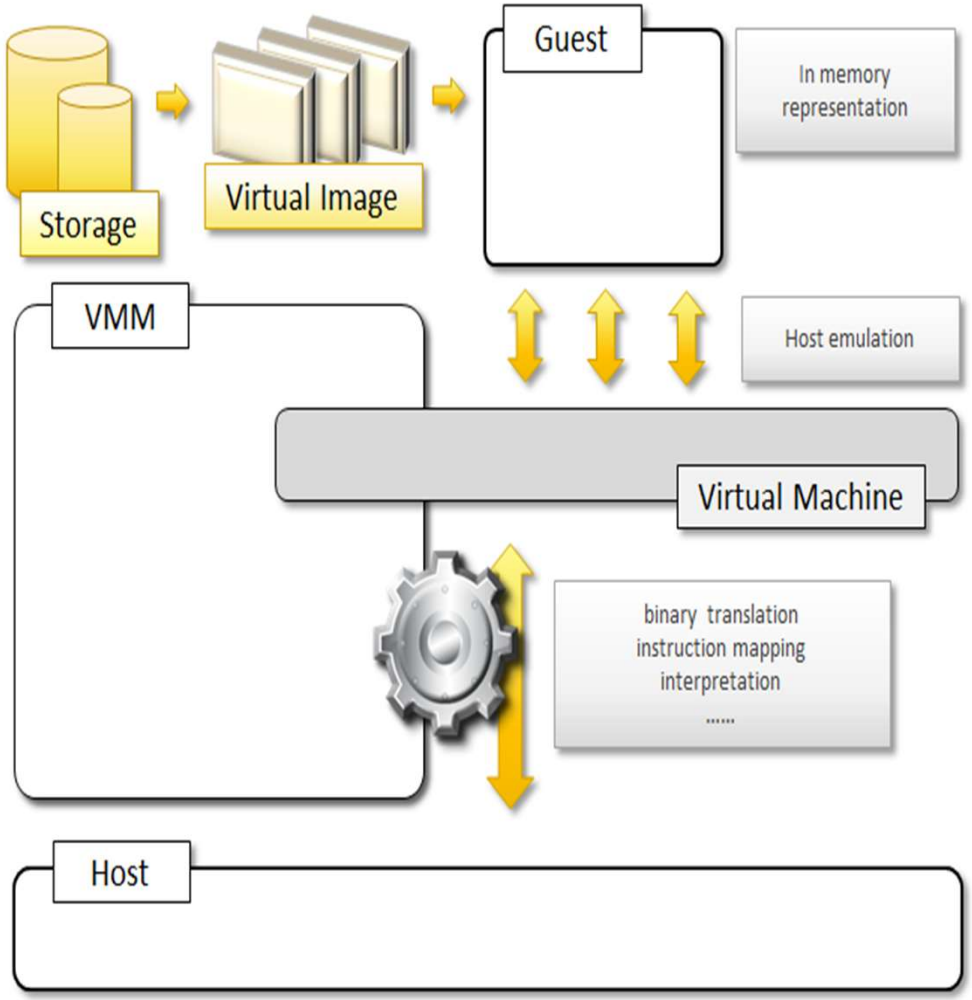
Security Rings and Privileged Modes

- Machine reference model also provides ways for **implementing a minimal security model for managing and accessing shared resources.**
- For this purpose, the instruction set exposed by the hardware has been divided **into different security classes, which define who can operate with them**.
- **The first** distinction can be made between ***privileged*** **and** ***non-privileged*** **instructions.**
- **Non-privileged instructions** are those instructions that can be **used without interfering with other tasks because they do not access shared resources**. This category contains, for example, all the floating, fixed point, and arithmetic instructions.
- **Privileged instru**ctions are those that are executed under specific restrictions and are mostly used for **sensitive operations, which** expose (*behavior sensitive*) **or modify** (*control sensitive*) the privileged state.
- a possible implementation features a hierarchy of privileges (see **Figure**) in the form of ring based security: *Ring 0, Ring 1, Ring 2,* and *Ring 3;* **Ring 0** is in the **most privileged level** and the **Ring 3** in the **least privileged level. Ring 0** is used by **the kernel** of the OS and rings 1 and 2 are used by the OS level services and **Ring 3** is used by **the user**. Recent systems support only two levels with Ring 0 for the supervisor mode and Ring 3 for user mode.



Hardware Level Virtualization

- Hardware level virtualization is a virtualization technique that provides an **abstract execution environment in terms of computer hardware** on top of which a **guest operating system can be run**.
- In this model, **the guest is represented by the operating system, the host by the physical computer hardware, the virtual machine by its emulation, and virtual machine manager by the *hypervisor*.**
- **The hypervisor** is **generally a program, or a combination of software and hardware**, that allows the abstraction of the underlying physical hardware.
- Hardware level virtualization is also called ***system virtualization***, since it provides ISA to virtual machines, which is the representation of the hardware interface of a system. This is to differentiate from *process virtual machines*, which expose ABI to virtual machines.



Hypervisors

- A fundamental element of hardware virtualization is the **hypervisor, or virtual machine manager** (VMM). It **recreates a hardware environment, where guest operating systems are installed.** There are two major types of hypervisors: *Type I* and *Type II*.
- ***Type I*** hypervisors **run directly on top of the hardware**. Therefore, they take the place of the operating systems and interact directly with the ISA interface exposed by the underlying hardware, and emulate this interface in order to allow the management of guest operating systems. This type of hypervisors is also called *native virtual machine*, since it run natively on hardware.
- ***Type II*** hypervisors **require the support of an operating system** to provide virtualization services. This means that they are programs managed by the operating system, which interact with it through the ABI and emulate the ISA of virtual hardware for guest operating systems. This type of hypervisors is also called *hosted virtual machine*, since it is hosted within an operating system.

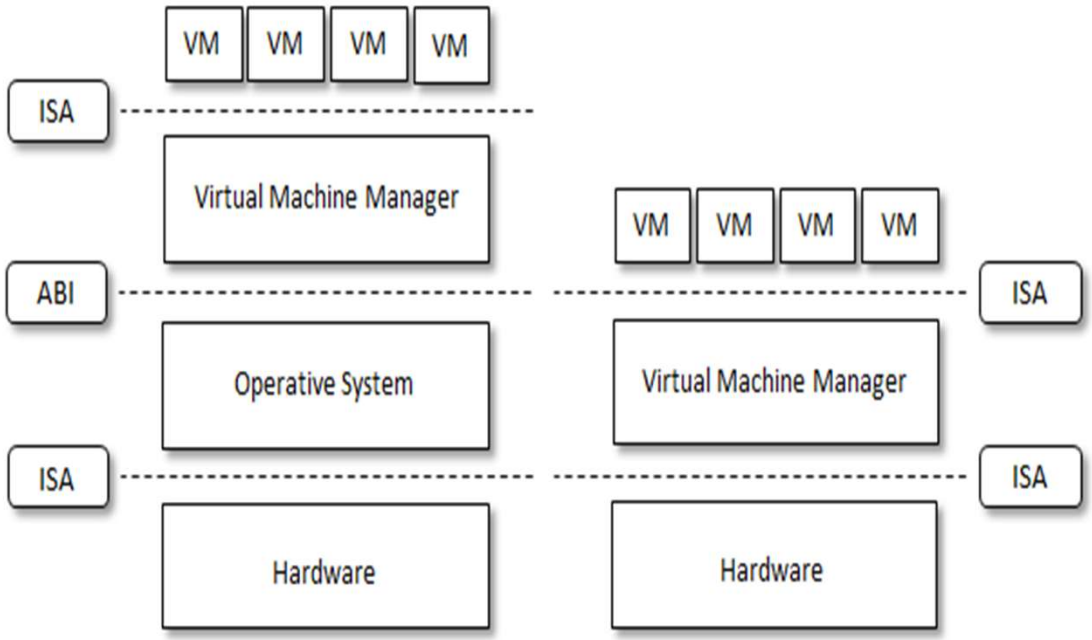


Fig-Hosted (left) and Native (right) Virtual Machine

Hypervisor Reference Architecture

- Conceptually, a virtual machine manager is internally organized as described in the Figure.
- **Three main modules coordinate** their activity in order to emulate the underlying hardware: ***dispatcher***, ***allocator***, **and** ***interpreter***.
- The **dispatcher** constitutes the **entry point of the monitor and reroutes the instructions** issued by the virtual machine instance to one of the two other modules.
- The **allocator** is **responsible for deciding the system resources to be provided to the VM**: whenever a virtual machine tries to execute an instruction that results in changing the machine resources associated with that VM, the allocator is invoked by the dispatcher.
- The **interpreter** module consists of **interpreter routines**. These are executed **whenever a virtual machine executes a privileged instruction:** a trap is triggered and the corresponding routine is executed.

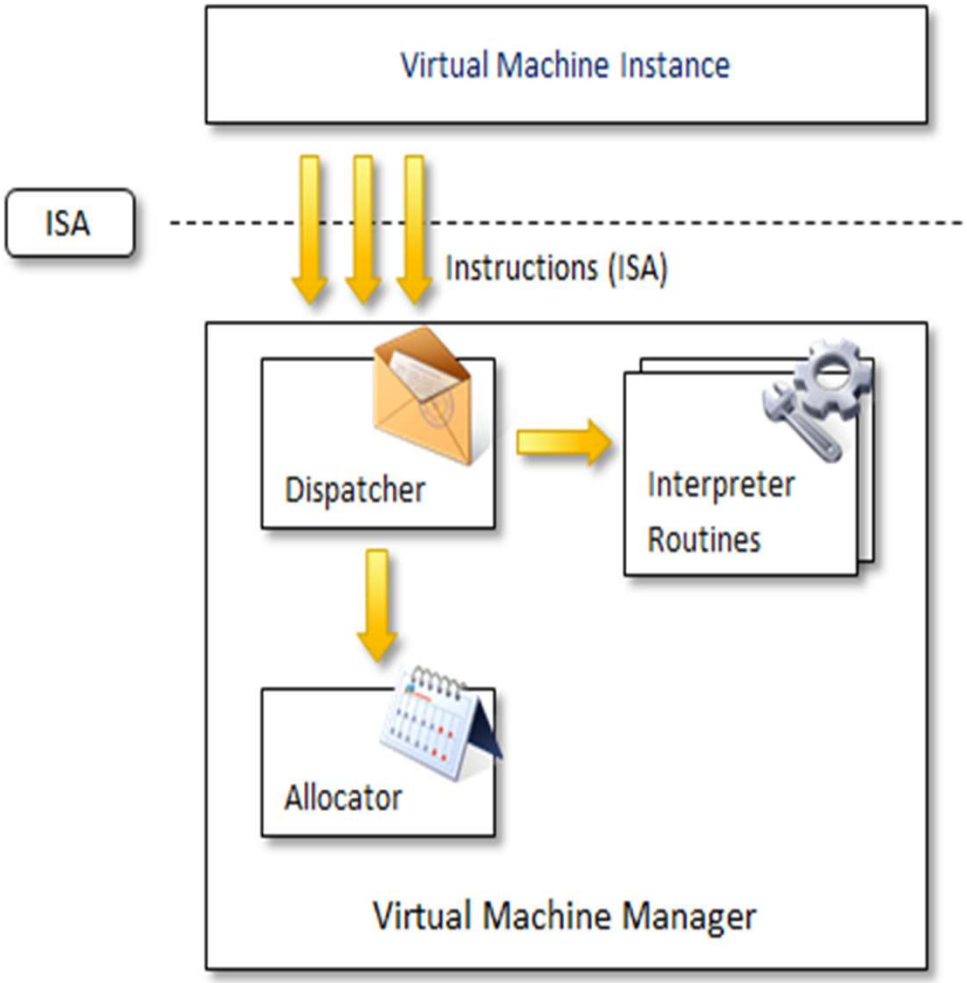


Fig- hypervisor reference architecture

Hypervisor Reference Architecture *contd…*

- The design and architecture of a virtual machine manager, together with the underlying hardware design of the host machine, determine the full realization of hardware virtualization, where a guest operating system can be transparently executed on top of a VMM as if it was run on the underlying hardware.
- The criteria that need to be met by a virtual machine manager to efficiently support virtualization were established by Goldberg and Popek in 1974 [23]. Three properties have to be satisfied:
  - *Equivalence:* a guest running under the control of a virtual machine manager should exhibit the same behavior that when it is executed directly on the physical host.
  - *Resource control:* the virtual machine manager should be in complete control of virtualized resources.
  - *Efficiency:* a statistically dominant fraction of the machine instructions should be executed without intervention from the virtual machine manager.
- The major factor that determines whether these properties are satisfied is represented by the layout of the ISA of the host running a virtual machine manager. **Popek and Goldberg** provided a classification of the instruction set and proposed **three theorems** that define the properties that hardware instructions need to satisfy in order to efficiently support virtualization.

Hardware Virtualization Techniques

- Hardware Assisted Virtualization.
- Full Virualization
- ParaVirtualization
- Partial Virtualization

Hardware Virtualization Techniques

- **Hardware-assisted virtualization:**
  - This term refers to a scenario in which the hardware provides architectural support for building a virtual machine manager able to run a guest operating system in complete isolation.
  - This technique was originally introduced in the IBM System/370. At present, examples of **hardware-assisted virtualization are the extensions to the x86-64 bit architecture introduced with *Intel VT***
- **Full virtualization**
  - Full virtualization refers to the ability of **running a program**, most likely an operating system, on top of a virtual machine directly and without any modification, as if it were run on the raw hardware.
  - In order to make this possible, virtual machine managers are required to provide a **complete emulation of the entire underlying hardware.**
  - The principal advantage of full virtualization is **complete isolation**, which leads to enhanced security, ease of emulation of different architectures, and coexistence of different systems on the same platform. Whereas it is a desired goal for many virtualization solutions, it poses important concerns on performance and technical implementation.

Hardware Assisted Virtualization

```
+-----------------------+
|     Applications      |
+-----------------------+
         |
         ↓
+-----------------------+
|     Guest OS          |
| (Unmodified)          |
+-----------------------+
         |
         ↓
+-----------------------+
|     Hypervisor        |
| (VM Monitor)          |
+-----------------------+
         |
         ↓
+-----------------------+
|  Hardware with VT     |
| CPU (Intel VT-x /     |
| AMD-V)                |
+-----------------------+
```

Advantages

✅ No need to modify the guest OS

✅ Supports **any operating system** (Linux, Windows, etc.)

✅ Better **performance** than software virtualization

✅ Strong **isolation and security**

Disadvantages

❌ Requires modern CPUs with virtualization support

❌ Slight overhead compared to native execution

Full Virtualization in Cloud Computing

```
+-----------------------+
|     Applications      |
+-----------------------+
         |
         ↓
+-----------------------+
|    Guest OS           |
| (Unmodified)          |
+-----------------------+
         |
         ↓
+-----------------------+
|     Hypervisor        |
| (VMM)                 |
+-----------------------+
         |
         ↓
+-----------------------+
|     Hardware          |
| CPU | Memory | I/O    |
+-----------------------+
```

Advantages

✅ Supports **any operating system**
✅ Strong **isolation** between virtual machines
✅ Easy **VM migration and management**
✅ Widely used in cloud environments

Disadvantages

❌ More **overhead** than paravirtualization
❌ Performance slightly lower without hardware support
❌ Requires a powerful hypervisor

Examples

- **VMware Workstation / ESXi**
- **Oracle VirtualBox**

Hardware Virtualization Techniques

- **Paravirtualization**
  - This is a **not transparent** virtualization solution that allows **implementing thin virtual machine** managers.
  - Paravirtualization techniques **expose a software interface** to the virtual machine that is slightly modified from the host and, as a consequence, guests need to be modified.
  - The aim of paravirtualization is to **provide the capability to demand the execution of performance critical operation** directly on the host.
  - solutions using paravirtualization include: *VMWare*, *Parallels*, and some solutions for embedded and real time environment such as *TRANGO*, *Wind River*, and *XtratuM*.
- **Partial virtualization**
  - Partial virtualization provides **a partial emulation of the underlying** hardware, thus not allowing the **complete execution of the guest operating** system in complete isolation.
  - Partial **virtualization allows many applications to run transparently** but not all the features of the operating system can be supported as happens with full virtualization.
  - An example of partial virtualization is address space virtualization used in time sharing systems.
  - Partial virtualization was implemented on the experimental *IBM M44/44X*. Address space virtualization is a common feature of contemporary operating systems.

ParaVirtualization

```
+-----------------------+
|     Applications      |
+-----------------------+
          |
          ↓
+-----------------------+
|  Modified Guest OS    |
| (Paravirtualized OS)  |
|  - Hypercalls         |
+-----------------------+
          |
          ↓
+-----------------------+
|     Hypervisor        |
| (Xen, PV Hypervisor)  |
+-----------------------+
          |
          ↓
+-----------------------+
|      Hardware         |
|  CPU | Memory | I/O   |
+-----------------------+
```

Paravirtualization is a **virtualization technique in cloud computing** where the **guest operating system (OS) is modified** so that it is aware it is running inside a virtual machine and **communicates directly with the hypervisor** for certain operations.

Advantages

✅ Better **performance** than full virtualization
✅ Lower **CPU overhead**
✅ Faster **I/O operations**
✅ Efficient **resource utilization**

Disadvantages

❌ Requires **modifying the guest OS**
❌ Cannot run **unmodified operating systems** (e.g., standard Windows)
❌ Less flexible than full virtualization

Examples of Paravirtualization

- **Xen Hypervisor** (classic paravirtualized mode)
- **Linux PV drivers** in cloud platforms

Partial Virtualization

- **Partial Virtualization** in **cloud computing** is a virtualization technique where **only some parts of the hardware are virtualized**, while other parts are accessed **directly by the operating system**.
- In this approach, the **guest operating system is aware that it is running in a virtualized environment** and may need **modifications** to work correctly.

- 🔹 **Example**
- If:
- CPU and memory are virtualized
- Network card and disk are accessed directly
- ➡️ This setup is **partial virtualization**.

🔹 **Advantages**

- ✅ Better performance than full virtualization
  ✅ Reduced overhead
  ✅ Faster execution
  ✅ Efficient resource utilization

🔹 **Disadvantages**

- ❌ Guest OS must be modified
  ❌ Limited OS compatibility
  ❌ Less isolation than full virtualization

Assignment Questions

- 1. What is cloud computing. Explain different major models used for deployment in cloud computing? Give the benefits of cloud computing?
- 2. Explain IaaS, PaaS and SaaS with suitable examples in detail.
- 3.Explain the differences between Parallel computing and Distributed computing. Give example for each difference in detail.
- 4. Explain about different technologies used in distributed computing with one example for each?
- 5. What is virtualization. Explain different virtualization techniques in detail with examples?

Operating System Level Virtualization

- Operating System level virtualization offers the opportunity **to create different and separated execution environment for applications** that are managed concurrently.
- Differently from hardware virtualization, **there is no virtual machine manager or hypervisor** and the virtualization is done within a single operating system, where the **OS kernel allows for multiple isolated** user space instances.
- **The kernel is also responsible for sharing the system resources** among instances and for limiting the impact of instances on each other.
- A user space instance in general contains **a proper view of the file system** which is completely isolated, separate IP addresses, software configurations, and access to devices.
- **Examples** of operating system level virtualizations are: *FreeBSD Jails*, *IBM Logical Partition (LPAR)*, *SolarisZones* and *Containers*, *Parallels Virtuozzo Containers*, *OpenVZ*, *iCore Virtual Accounts*, *Free Virtual Private Server (FreeVPS)* and others.
- The services offered by each of these technologies differ and most of them are available on Unix based systems.

- **Operating System–level Virtualization** is a virtualization technique where a single OS kernel allows multiple isolated user-space environments (called **containers**) to run on the same system.
  All containers share the **same kernel**, but each behaves like an independent system with its own processes, files, and network.
  It is **lightweight, fast**, and uses fewer resources compared to full virtualization.
- **Example:**
  Docker running multiple containers (e.g., one container for a web server, another for a database) on the same Linux OS using the same kernel.

Programming Language Level Virtualization

- Programming language level virtualization is mostly used for achieving **ease of deployment of applications, managed execution, and portability** across different platforms and operating systems.
- It consists of a virtual machine executing **the byte code of a program**, which is the result of the compilation process.
- **Compilers implemented** used this technology produce a **binary format** representing the machine code for an abstract architecture.
- The main advantage of programming-level virtual machines, also called process virtual machines, is the ability of providing a **uniform execution environment across different platforms.**
- Programs compiled into byte code can be executed on any operating system and platform for which a virtual machine able to execute that code has been provided.
- As an example, both Java and .NET provide an infrastructure for pluggable security policies and code access security frameworks.

- **Programming Language–level Virtualization** provides a virtual execution environment created by a **language runtime**, rather than by the operating system or hardware.
  Programs written in that language run on a **virtual machine or interpreter**, which abstracts the underlying OS and hardware.
  This improves **portability, security, and memory management**.
- **Example:**
  Java Virtual Machine (JVM) allows Java programs to run on any platform (Windows, Linux, macOS) without modification.

Application Level Virtualization

- Application level virtualization is a technique **allowing applications to be run on run-time environments** which do not natively support all the features required by such applications.
- In this scenario, applications are **not installed** in the expected run time environment, but run as if they were.
- In general, these techniques are mostly concerned with partial file systems, libraries, and operating system component emulation.
- Such emulation is performed by a thin layer—a program or an operating system component—that is in charge of executing the application.
-

- **Application-level Virtualization** allows an application to run in an isolated virtual environment without being fully installed on the host operating system.
  The application is separated from the OS and other applications, reducing conflicts and dependency issues.
  It improves **portability, security, and ease of deployment**.
- **Example:**
  Microsoft App-V or **Citrix Virtual Apps**, where applications run virtually without traditional installation.

Application Level Virtualization

- Emulation can also be used to **execute program binaries compiled for different hardware architectures**. In this case, one of the following strategies can be implemented:
  - ***Interpretation.*** In this technique every source instruction is interpreted by emulator for executing native ISA instructions leading to poor performance. Interpretation has a minimal startup cost but a huge overhead since each instruction is emulated.
  - ***Binary Translation****.* In this technique every source instruction is converted to native instructions with equivalent functions. After a block of instructions is translated it is cached and reused. Binary translation has a large initial overhead cost but over time it is subject to a better performance, since previously translated instruction blocks are directly executed.
- Application virtualization is **a good solution in the case of missing libraries** in the host operating system: in this case a replacement library can be linked with the application or library calls can be remapped to existing functions available in the host system.
- Another advantage is that in this case the virtual machine manager is much lighter since it provides a partial emulation of the run time environment if compared to hardware virtualization.
- One of the most popular solution implementing application virtualization is ***Wine***, which is a software application allowing Unix-like operating systems to execute programs written for the Microsoft Windows platform.

Storage Virtualization

- Storage virtualization is a system administration practice that allows decoupling the **physical organization** of the hardware from **its logical representation.**
- By using this technique users do **not have to be worried about the specific location of their data**, which can be identified by using a logical path.
- Storage virtualization **allows harnessing a wide range of storage** facilities and representing them under a single logical file system.
- There are different techniques for storage virtualization one of the most popular includes network based virtualization by means of ***Storage Area Networks (SANs)*.**
- Storage Area Networks use a network accessible device through a large bandwidth connection to provide storage facilities.

- **Storage Virtualization** is a technique that abstracts multiple physical storage devices into a single logical storage pool.
  It hides the complexity of physical storage and allows flexible, centralized storage management. This improves **scalability, utilization, and data management efficiency**.
- **Example:**
  SAN (Storage Area Network) using technologies like **VMware vSAN** that combine multiple disks into one virtual storage system.

Network Virtualization

- Network virtualization **combines hardware appliances and specific software** for the creation and management of a virtual network.
- Network virtualization can **aggregate different physical networks into a single logical network** (*external* network virtualization) or provide network like functionality to an operating system partition (*internal* network virtualization).
- The result of external network virtualization is generally a ***Virtual LAN*** *(VLAN)*. A *VLAN* is an aggregation **of hosts that communicate** with each other as if they were located under the same broadcasting domain.
- Internal network virtualization is generally applied together with hardware and operating system level virtualization in which the guests obtain a virtual network interface to communicate with.
- There are several options for implementing internal network virtualization: the guest can share the same network interface of the host and use NAT to access the network; the virtual machine manager can emulate, and install on the host, an additional network device together with the driver; or the guest can have a private network only with the guest.

- **Network Virtualization** is the process of creating multiple virtual networks on top of a single physical network infrastructure.
  It abstracts network resources such as switches, routers, and links, allowing flexible and isolated network configurations.
  This improves **network scalability, security, and efficient resource utilization**.
- **Example:**
  VLANs or **Software Defined Networking (SDN)** used in data centers to create multiple virtual networks over the same physical hardware.

Desktop Virtualization

- Desktop virtualization abstracts the **desktop environment** available on a personal computer in order to provide access to it by using a client server approach.
- Desktop virtualization provides the same outcome of hardware virtualization but serves a different purpose.
- Similarly to hardware virtualization it makes accessible a different system as if it was natively installed on the host, but this system is remotely stored on a different host and accessed through a network connection.
- Moreover, desktop virtualization addresses the problem of making the same desktop environment accessible from everywhere.
- While the term desktop virtualization strictly refers to the ability to remotely access a desktop environment, generally, the desktop environment is stored in a remote server or a data center which provides a high availability infrastructure and ensures the accessibility and the persistence of the data.
- The basic services for remotely accessing a desktop environment are implemented in software components such as: *Windows Remote Services*, *VNC*, and *X Server*.
- Infrastructures for desktop virtualization based on Cloud computing solutions are: *Sun Virtual Desktop Infrastructure (VDI)*, *Parallels Virtual Desktop Infrastructure (VDI)*, *Citrix XenDesktop* and others.

- **Desktop Virtualization** is a technology that allows desktop environments to run on a centralized server instead of a local machine.
  Users access their virtual desktops remotely using thin clients or personal devices.
  This improves **security, centralized management, and cost efficiency**.
- **Example:**
  Virtual Desktop Infrastructure (VDI) using **VMware Horizon** or **Citrix Virtual Desktops**, where users access desktops hosted on a data center server.

Application Server Virtualization

- Application server virtualization abstracts a collection of application servers that provide the same services as a single virtual application server by using load balancing strategies and providing a high availability infrastructure for the services hosted in the application server.
- This is a particular form of virtualization and serves the same purpose of storage virtualization: providing a better quality of service rather than emulating a different environment.

- **Application Server Virtualization** allows multiple application server instances to run independently on the same physical or virtual server.
  Each application server is isolated, ensuring better performance, scalability, and fault tolerance.
  It helps organizations efficiently utilize server resources and simplify application deployment.
- **Example:**
  Running multiple **Apache Tomcat** or **JBoss/WildFly** application servers as virtual instances on a single server using **VMware** or **Hyper-V**.

Virtualization and Cloud Computing

- Virtualization **plays an important role** in Cloud computing, since it allows for the appropriate degree of customization, security, isolation.
- Virtualization technologies are **primarily used** to offer configurable computing environments and storage.
- Particularly important is the role of virtual computing environment and execution virtualization techniques. Among these, hardware and programming language virtualization are the techniques adopted in Cloud computing systems.
- virtualization also gives the opportunity of designing more efficient computing systems by means of consolidation
- Server consolidation and virtual machine migration are principally used in case of hardware virtualization even though technically possible also in case of programming language virtualization.
- Storage virtualization constitutes an interesting opportunity given by virtualization technologies, often complimentary to the execution virtualization.
- Finally, Cloud computing revamps the concept of desktop virtualization, initially introduced in the mainframe era.

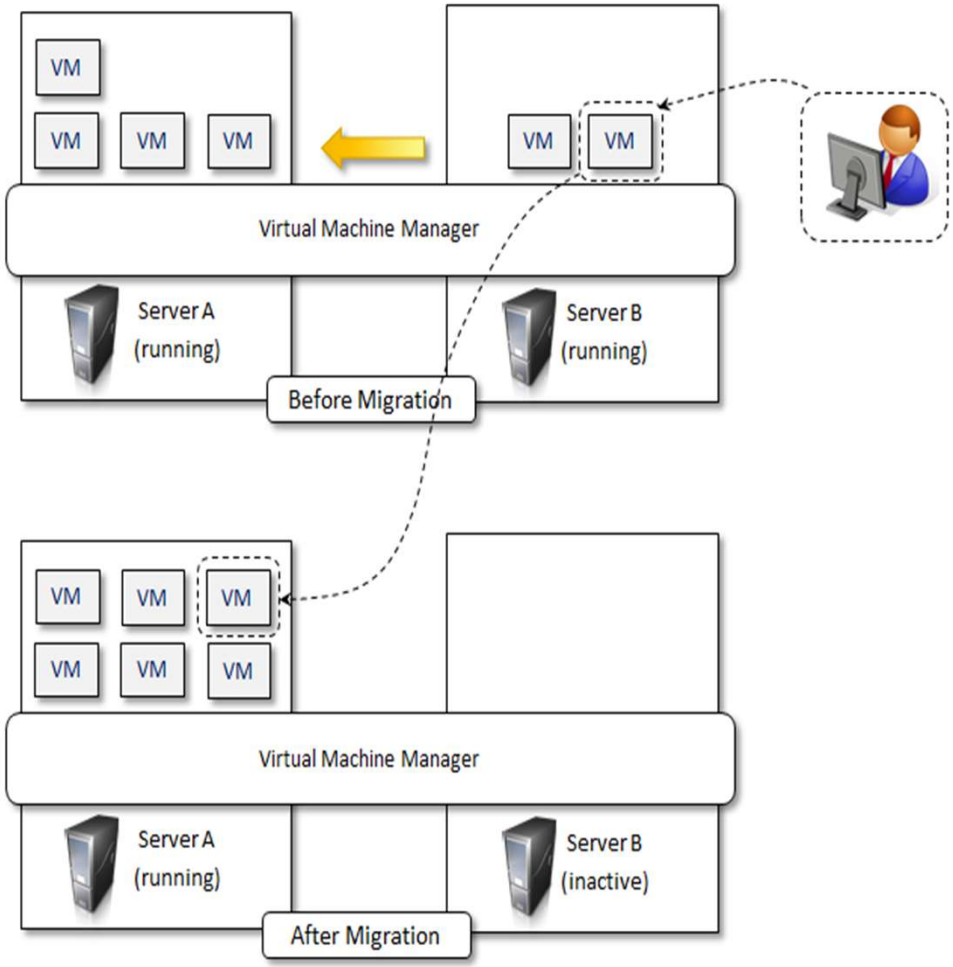


Fig- . Live Migration and Server Consolidation

Pros and Cons of Virtualization

- ***Advantages of Virtualization***
  - Managed execution and isolation are perhaps the most important advantages of virtualization.
- Managed execution refers to the cloud provider's responsibility to **control, monitor, and manage the execution of applications and workloads**, including resource allocation, scaling, fault handling, and security, without user intervention.
- **Example:**
  In **AWS Lambda**, the cloud provider automatically manages server provisioning, execution, scaling, and fault tolerance while running user code.
- **Isolation**

Isolation in cloud computing ensures that **multiple users or applications share the same physical resources securely**, without interfering with each other's data or performance.

- **Example:**
  In **virtual machines (VMs)** hosted on a single physical server, each VM is isolated using a **hypervisor**, so one user's VM cannot access another user's VM.

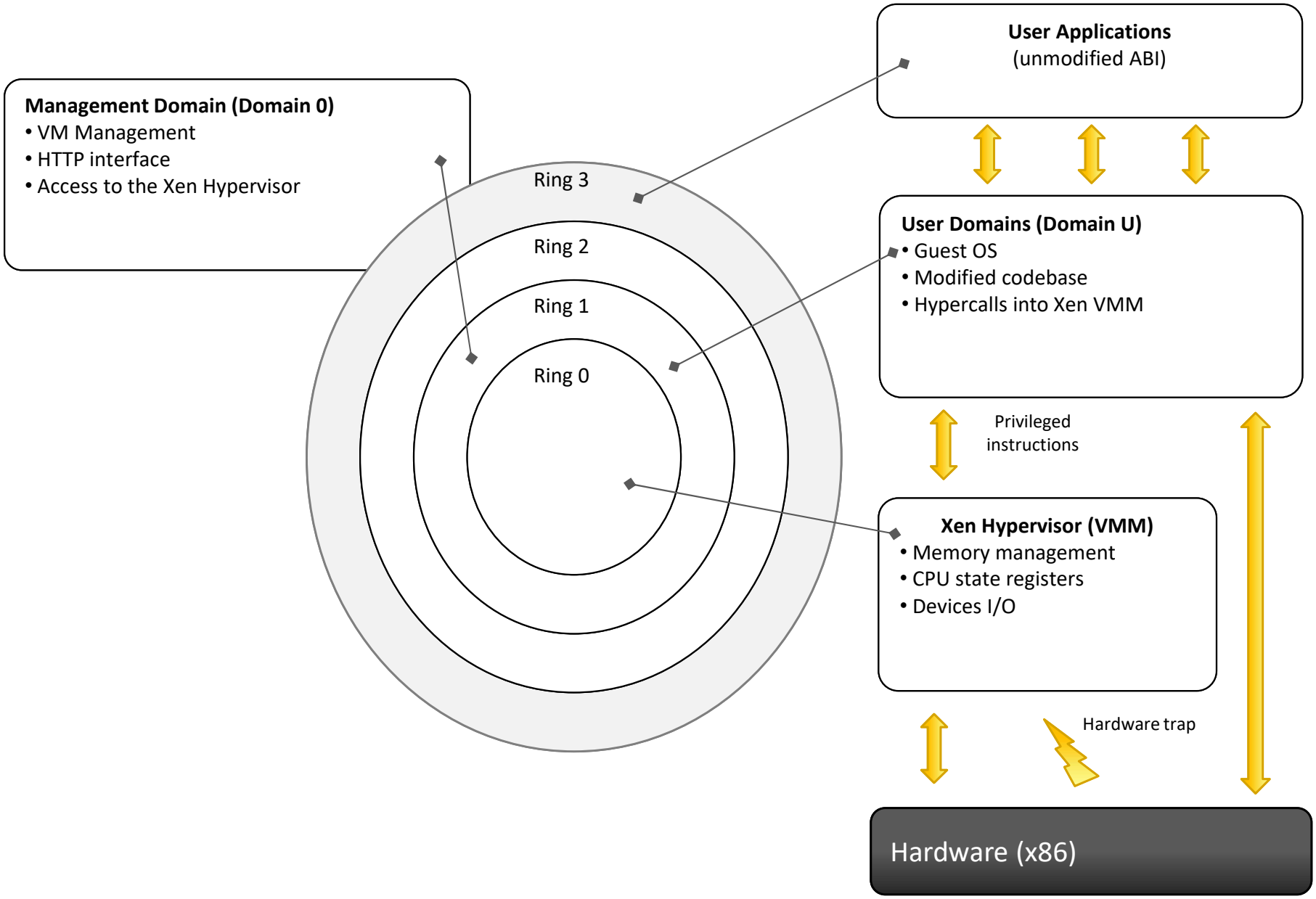
- these two characteristics allow building secure and controllable computing environments. A virtual execution environment can be configured as a sandbox, thus preventing any harmful operation to cross the borders of the virtual host.
- Moreover, allocation of resources and their partitioning among different guests is simplified, being the virtual host controlled by a program.
- Portability is another advantage of virtualization, especially for execution virtualization techniques.
- Portability and self-containment also contribute to reduce the costs for maintenance, since the number of hosts is expected to be lower than the number of virtual machine instances.
- Finally, by means of virtualization it is possible to achieve a more efficient use of resources. Multiple systems can securely coexist and share the resources of the underlying host, without interfering with each other.

Pros and Cons of Virtualization

- ***Disadvantages of Virtualization***
  - ***Performance Degradation***
    - Performance is definitely one of the major concerns when using virtualization technology. Since virtualization interposes an abstraction layer between the guest and the host, increased latencies and delays can be experienced by the guest.
    - Also, when hardware virtualization is realized through a program that is installed or executed on top of the host operating systems, a major source of performance degradation is represented by the fact that the virtual machine manager is executed and scheduled together with other applications, thus sharing with them the resources of the host.
  - ***Inefficiency and Degraded User Experience***
    - Virtualization can sometime led to an inefficient use of the host. In particular, some of the specific features of the host cannot be exposed by the abstraction layer and then become not accessible.
  - ***Security Holes and New Threats***
- Virtualization opens the door to a new and unexpected form of *phishing*. The capability of emulating a host in a complete transparent manner, has led the way to malicious programs which are designed to extract from the guest sensitive information.

TECHNOLOGY EXAMPLES

- In this section, we discuss the most relevant technologies and approaches utilized in the field.
  - Xen: Paravirtualization
  - VMware: Full Virtualization
  - Microsoft Hyper-V


User Applications
(unmodified ABI)
Management Domain (Domain 0)
• VM Management
• HTTP interface
• Access to the Xen Hypervisor
Ring 3
Ring 2
Ring 1
Ring 0
User Domains (Domain U)
• Guest OS
• Modified codebase
• Hypercalls into Xen VMM
Privileged
instructions
Xen Hypervisor (VMM)
• Memory management
• CPU state registers
• Devices I/O
Hardware trap
Hardware (x86)

Contemporary Virtualization Techniques

- Virtualization techniques are a set of computing technologies that allow users to share resources without changing the underlying hardware or software. Some contemporary virtualization techniques include:

Contemporary Virtualization Techniques

- **Desktop virtualization**
- Allows multiple virtual desktops to run on a single physical computer or server. This is useful for remote work and provides a secure user experience.
- **Server virtualization**
- Allows multiple virtual servers to run on a single physical server. This can help maximize resource utilization and reduce hardware costs.
- **Storage virtualization**
- Allows users to access and manage all storage devices in a network as a single pool. This can help simplify storage provisioning for virtual machines.
- **Application virtualization**
- Allows users to create virtual instances of applications needed for business operations. This can help simplify software management and reduce conflicts between applications.

Contemporary Virtualization Techniques

- **Network virtualization**
- Allows users to share resources such as servers, storage, and networking without changing the underlying hardware or software.
- **GPU virtualization**
- Allows a physical GPU to be shared across multiple virtual machines. This can be useful for tasks that require high-performance graphics.
- **Cloud virtualization**
- Allows organizations to efficiently allocate resources, scale their operations, and provision virtualized environments on demand.

virtualization different from cloud computing?

- Cloud computing is the **on-demand delivery of computing resources over the internet** with pay-as-you-go pricing. Instead of buying, owning, and maintaining a physical data center, you can access technology services, such as computing power, storage, and databases, as you need them from a cloud provider.
- Virtualization technology makes cloud computing possible. **Cloud providers set up and maintain their own data centers**. They create different virtual environments that use the underlying hardware resources. You can then program your system to access these cloud resources by using APIs. Your infrastructure needs can be met as a fully managed service.

How is server virtualization different from containerization?

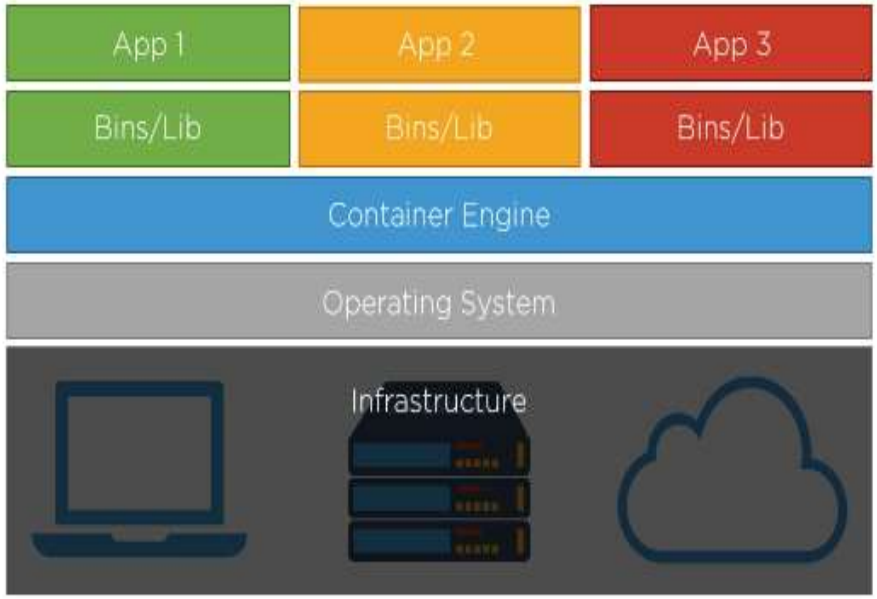
- Containerization is a **way to deploy application code to run on any physical or virtual environment without changes.** Developers bundle application code with related libraries, configuration files, and other dependencies that the code needs to run. This **single package of the software, called a container, It c**an run independently on any platform. Containerization is a type of application virtualization.
- You can think of server virtualization **as building a road to connect two places.** You have to **recreate an entire virtual environment and then run your** application on it. By comparison, containerization is like building a helicopter that can fly to either of those places. Your application is inside a container and can run on all types of physical or virtual environments.

What is Containers ?

- A container is a light, stand-alone, and executable software package that wraps an application with all its dependencies, such as libraries, configuration files, and binaries, in order to be run.
- Containers make sure that no matter where an application is deployed, whether it is in a developer's laptop, the testing environment, or the production server, it behaves the same way.

Containers

- Containers do this by isolating an application from the underlying system, and therefore they are not dependent on any setting or software installed in the operating system of the host. Instead everything an application requires to run is packed within the container itself.
- This isolation allows containers to be more efficient compared to traditional virtual machines (VMs), which contain a full operating system besides the application.



Container

Primary Terminologies

- **Container**: An isolated, stand-alone unit that **encapsulates an application and all its dependencies**, it runs the same and consistently in any environment, independently of the host system, being unaffecting and not getting affected by it.
- **Docker**: Docker is an open-source platform designed **to make it easy for containers to be built, developed, and run**. It provides one with all the software required, in addition to development capabilities, to build, run, and manage containers for maximum efficiency.
- **Image**: A container image is **a lightweight, read-only, executable file** that includes everything needed to run a piece of software: the code, the runtime, the libraries, the environment variables, and configurations. It basically serves as a template for creating containers.
- **Containerization**: The way of bundling the application **together with all its dependencies** into a container .In this way, the application acts in the same way in which it is executed.
- **Orchestration**: It automatically takes care of the **coordination, scheduling, and management of multi-container deployments** running on a cluster of machines, container orchestration tools include Kubernetes and Docker Swarm.

Containers vs. Virtual Machines (VMs)

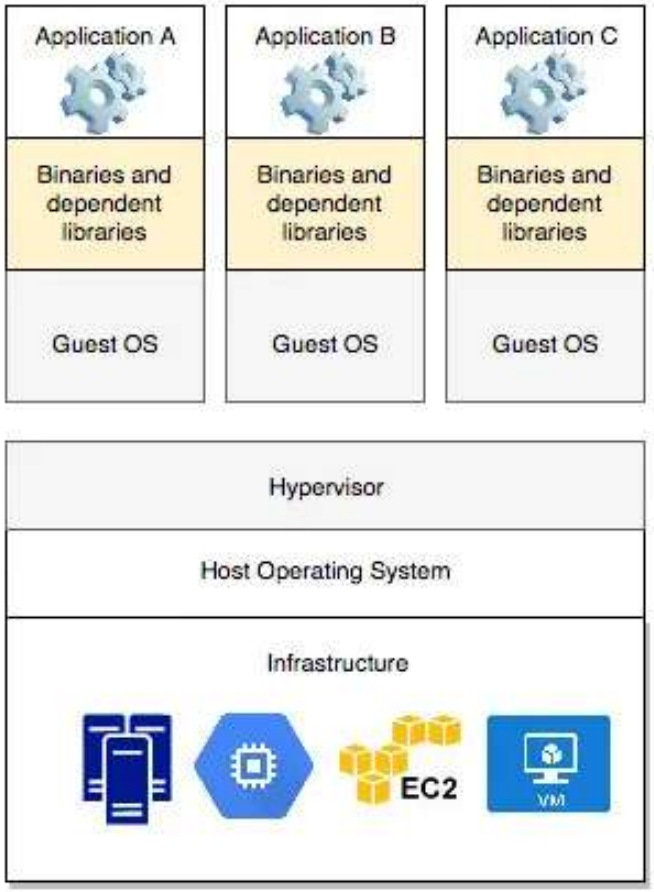
Containers

- **Architecture**: All containers share the host OS kernel; however, the running user spaces are isolated, making them lightweight.
- **Boot Time**: Containers have much less boot time typically in seconds, as they do not need to boot a full OS.
- **Isolation**: Containers provide isolation at the process level, which is less strong compared to VMs, but for many use cases this does not matter
- **Resource Usage**: Containers consume fewer resources because they do not need an entire OS—only the necessary binaries and libraries.

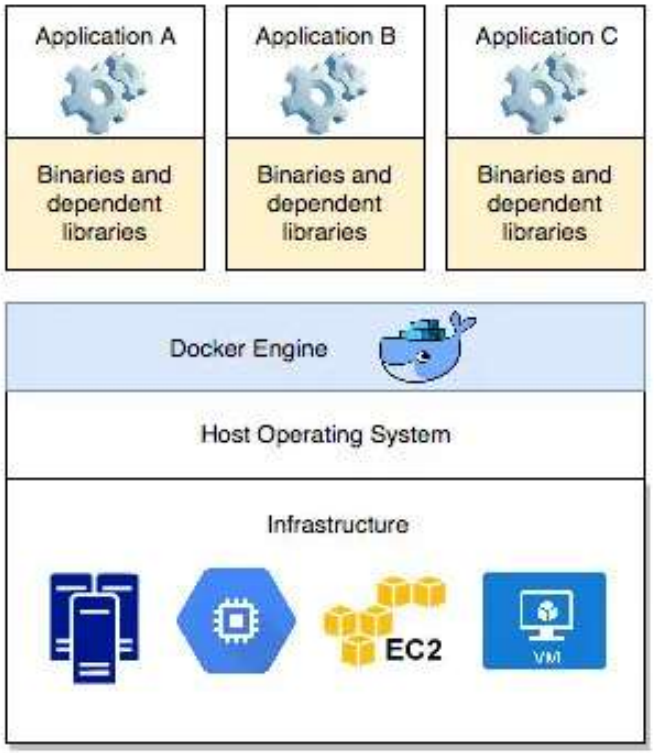
Containers vs. Virtual Machines (VMs)

- **Virtual Machines (VMs)**
- **Architecture**: A hypervisor that runs on the host OS includes a full guest OS with virtualized hardware.
- **Resource Usage**: Very high, as the full OS overhead is incurred for each instance.
- **Isolation**: Very good because each VM is a system on its own with its own OS.
- **Boot Time**: VMs typically have longer boot times because the full OS in a VM needs to be initialized.

Containers vs. Virtual Machines (VMs)



Virtual Machines based architecture



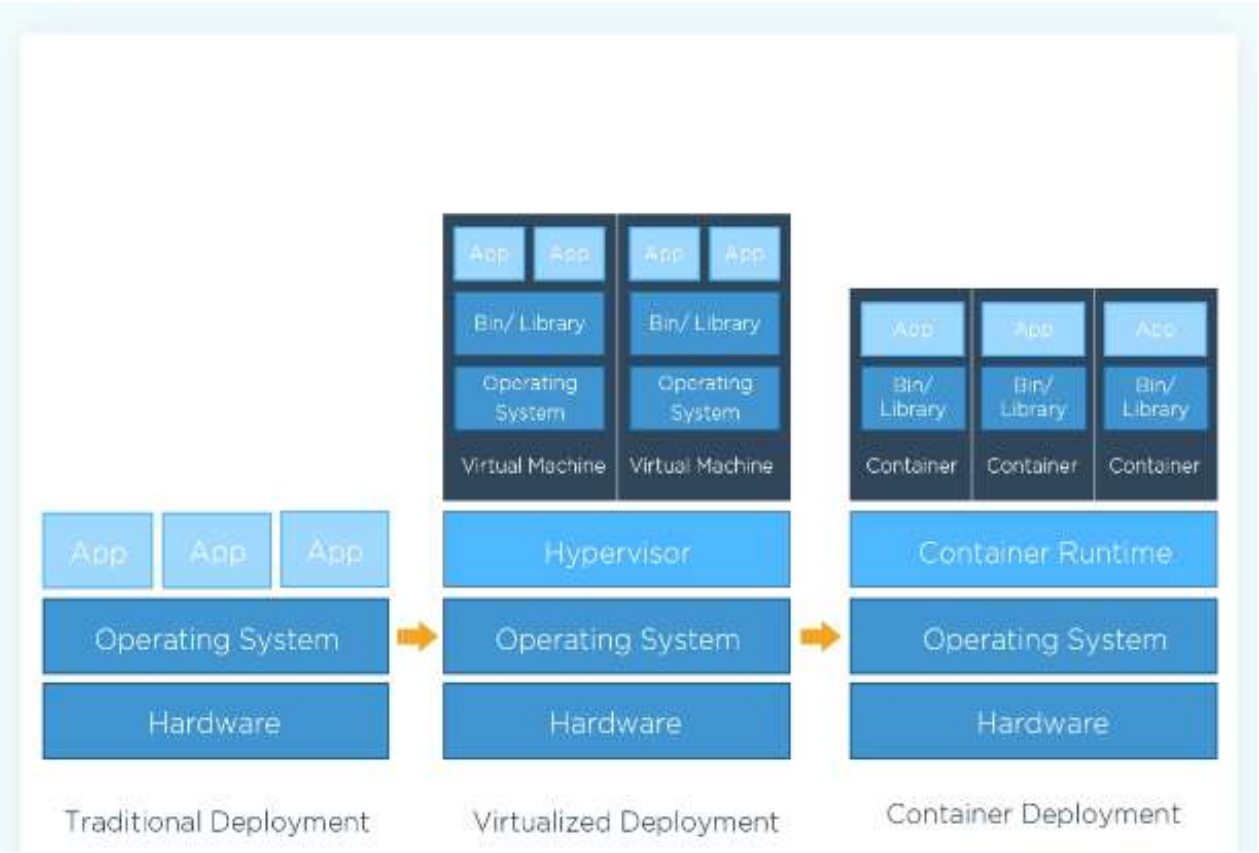
Container based architecture

Pablo Iorio

Virtual machines vs container based architectures

Containers vs. Virtual Machines (VMs)

- the differences between traditional deployment vs. virtualization vs. containerization



What is Containerization?

- Containerization is the **process of packing an application together with all its dependencies** into a container in order to allow the application to run consistently from one computing environment to another, in simple terms containerization involves using the **host OS kernel to run many isolated instances** of applications on the same machine, making it very lightweight and efficient in deploying applications.

Benefits of Containers

- **Portability:** Containers package all dependencies, enabling applications to run consistently across various environments.
- **Efficiency:** Containers use fewer resources than VMs, allowing for higher density of applications on the same hardware.
- **Scalability:** Containers can be easily scaled up or down, enabling efficient handling of dynamic workloads.
- **Faster Deployment:** Due to their lightweight nature, containers can be deployed quickly, enhancing development and CI/CD processes.
- **Consistency:** Containers ensure that the application runs the same way across development, testing, and production environments.

What is container orchestration?

- Container orchestration **automatically provisions, deploys, scales, and manages containerized applications** without worrying about the underlying infrastructure.
- Developers can implement **container orchestration anywhere** containers are, allowing them to automate the life cycle management of containers.

How does container orchestration work?

- Container orchestration architecture consists of **running container clusters across multiple machines and environments.** Each cluster typically consists of a group of nodes (also called server instances).
- **Worker nodes** run containers using container runtimes (such as Docker). A group of control plane nodes act as the orchestrator of the cluster.
- Users can **manage and monitor** containerized workloads with tools featuring application programming interfaces (APIs) and graphical user interfaces (GUIs).

How does container orchestration work? Conte....

- While there are **differences in methodologies** and capabilities across platforms and tools, container orchestration is essentially a **three-step process** (or cycle, when part of an iterative agile or DevOps pipeline):
- Provisioning
- Deployment
- Lifecycle management

How does container orchestration work? Conte....

1. Provisioning

- Define which container images make up the application and where they are located (in what container registry).
- Provision the containers with storage and other resources.
- Define and secure the network connections between containers.
- Specify versioning (for phased or canary rollouts).

2. Deployment

- The orchestration tool schedules the deployment of the containers (and replicas of the containers for resiliency) to a host. It chooses the best host based on available the central processing unit (CPU) capacity, memory or other requirements or constraints specified in the configuration file.

3. Lifecycle management

- Once the containers are deployed, the orchestration tool manages the lifecycle of the containerized application based on the container definition file (often a Dockerfile).
- Lifecycle management tasks include:
- Manage scalability (up and down), load balancing and resource allocation among the containers.
- Help ensure availability and performance by relocating the containers to another host if there is an outage or a shortage of system resources.
- Collect and store log data and other telemetry that monitor the health and performance of the application.

Deployment Strategies for Containerization Architecture

- Containerization architecture, popularized by platforms like Docker and Kubernetes, offers several deployment strategies to optimize resource utilization, scalability, and reliability. Here are some common deployment strategies:
- **1. Single Container Deployment**
- Simplest form where each service runs in its own container.
- Easy to set up and suitable for small-scale applications or services with minimal dependencies.
- **2. Multiple Containers per Host**
- Deploy multiple related services in separate containers on the same host.
- Enables better resource utilization by sharing the underlying infrastructure.
- Requires careful resource management to prevent contention.
- **3. Orchestration with Kubernetes**
- Utilizes Kubernetes to manage and orchestrate containers across a cluster of hosts.
- Kubernetes abstracts away the underlying infrastructure and handles tasks like scaling, load balancing, and service discovery.
- Offers declarative configuration and automated deployment, making it ideal for complex microservices architectures.

Deployment Strategies for Containerization Architecture

- **4. Service Mesh Deployment**
- Implements a dedicated infrastructure layer for handling service-to-service communication.
- Technologies like Istio or Linkerd are used to manage traffic, enforce policies, and provide observability.
- Ideal for microservices architectures with complex networking requirements.
- **5. Serverless Deployment**
- Abstracts away the infrastructure layer entirely, allowing developers to focus solely on writing code.
- Platforms like AWS Lambda, Azure Functions, and Google Cloud Functions execute code in response to events without provisioning or managing servers.
- Offers automatic scaling and pay-per-use pricing, making it cost-effective for sporadically-used services.
- **6. Blue/Green Deployment**
- Involves running two identical production environments (blue and green) simultaneously.
- Traffic is routed to one environment while the other remains idle.
- Allows for seamless updates with zero downtime by switching traffic between environments.

Types of container orchestration tools or Technologies

- Several container orchestrators are available on the market today.
- There are two types of container orchestration platforms:
- Docker is a container runtime environment, while Kubernetes is a container orchestration tool. Both are open-source technologies that help developers build, deliver, and scale containerized applications.
- **Docker:**
- It offers resources for building, sharing, and utilizing containers.
- Containers are created using Docker images, and the registry Docker Hub is used to share and store these images.
- Using a straightforward YAML file, Docker Compose enables you to define and manage multi-container applications.
- **Kubernetes:**
- Containerised application deployment, scalability, and management are automated by this robust field orchestration platform.
- It is capable of load balancing, rolling updates, self-healing, and automatic scaling.
- Kubernetes supports both on-premises and cloud settings and employs a declarative configuration methodology.

Introduction to Docker

- Docker is a powerful tool used for developing, packaging, and deploying applications efficiently.
- Docker is a container management service. Docker was released in 2013.
- It is open-source and available for different platforms like Windows, macOS, and Linux.
- Docker is quickly shipping, testing, and deploying code.

Introduction to Docker

- Docker is a set of Platforms as a service (PaaS) products that use Operating system-level virtualization to deliver software in packages called containers.
- Containers are isolated from one another and bundle their own software, libraries, and configuration files; they can communicate with each other through well-defined channels.
- All containers are run by a single operating system kernel and therefore use fewer resources than a virtual machine.

What is Docker?

- Docker is an open-source containerization platform by which you can pack your application and all its dependencies into a standardized unit called a container.
- Containers are light in weight which makes them portable and they are isolated from the underlying infrastructure and from each other container.
- You can run the docker image as a docker container in any machine where docker is installed without depending on the operating system.

Why Docker is popular?

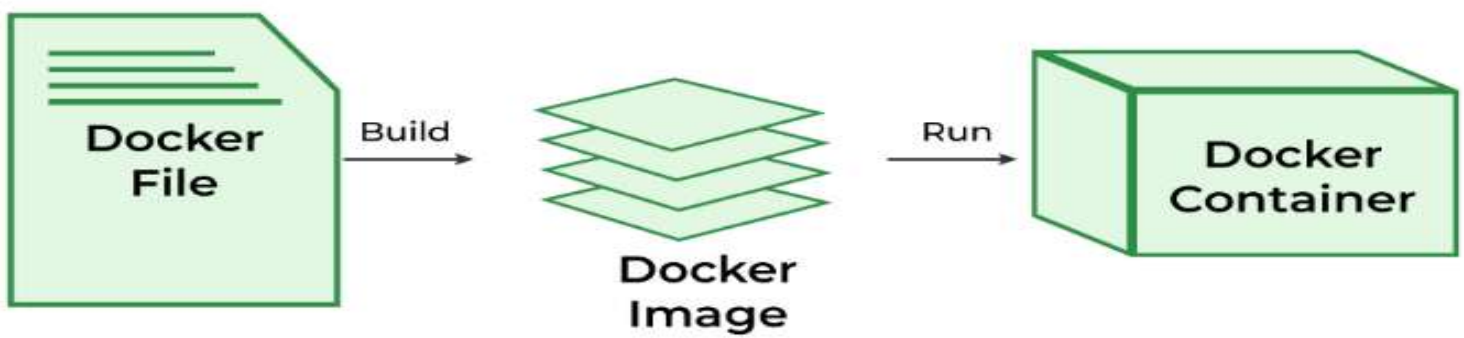
- Docker gained its popularity due to its impact on the software development and deployment. The following are the some of the main reasons for docker becoming popular:
- **Portability:** Docker facilitates the developers in packaging their applications with all dependencies into a single lightweight containers. It facilities in ensuring the consistent performance across the different computing environments.
- **Reproducibility:** Through encapsulating the applications with their dependencies within a container it ensures in software setups remaining consistent across the development, testing and production environments.
- **Efficiency:** Docker through its container based architecture it optimizes the resource utilization. It allows the developers to run the multiple isolated applications on a single host system.
- **Scalability:** Docker's scalability features facilitated the developers in making easier of their applications handling at time of workloads increment.

Key Components of Docker

- The following are the some of the key components of Docker:
- **Docker Engine:** It is a core part of docker, that handles the creation and management of containers.
- **Docker Image:** It is a read-only template that is used for creating containers, containing the application code and dependencies.
- **Docker Hub:** It is a cloud based repository that is used for finding and sharing the container images.
- **Dockerfile:** It is a script that containing instructions to build a docker image.
- **Docker Registry** : It is a storage distribution system for docker images, where you can store the images in both public and private modes.

What is a Dockerfile?

- The Dockerfile uses DSL (Domain Specific Language) and contains instructions for generating a Docker image. Dockerfile will define the processes to quickly produce an image. While creating your application, you should create a Dockerfile in order since the **Docker daemon** runs all of the instructions from top to bottom.
- (The Docker daemon, often referred to simply as “Docker,” is a background service that manages Docker containers on a system.)
- It is a text document that contains necessary commands which on execution help assemble a Docker Image.
- Docker image is created using a Dockerfile.



What is Docker Image?

- It is a file, comprised of multiple layers, used to execute code in a Docker container. They are a set of instructions used to create docker containers. **Docker Image is an executable package of software** that includes everything needed to run an application. This image informs how a container should instantiate, determining which software components will run and how.
- Docker Container is a virtual environment that bundles application code with all the dependencies required to run the application. The application runs quickly and reliably from one computing environment to another.

What is Docker Container?

- Docker container is a **runtime instance of an image.** Allows developers to package applications with all parts needed such as libraries and other dependencies. Docker Containers are runtime instances of Docker images.
- Containers contain the whole kit required for an application, so the application can be run in an isolated way.
- For eg.- Suppose there is an image of Ubuntu OS with NGINX SERVER when this image is run with the docker run command, then a container will be created and NGINX SERVER will be running on Ubuntu OS.

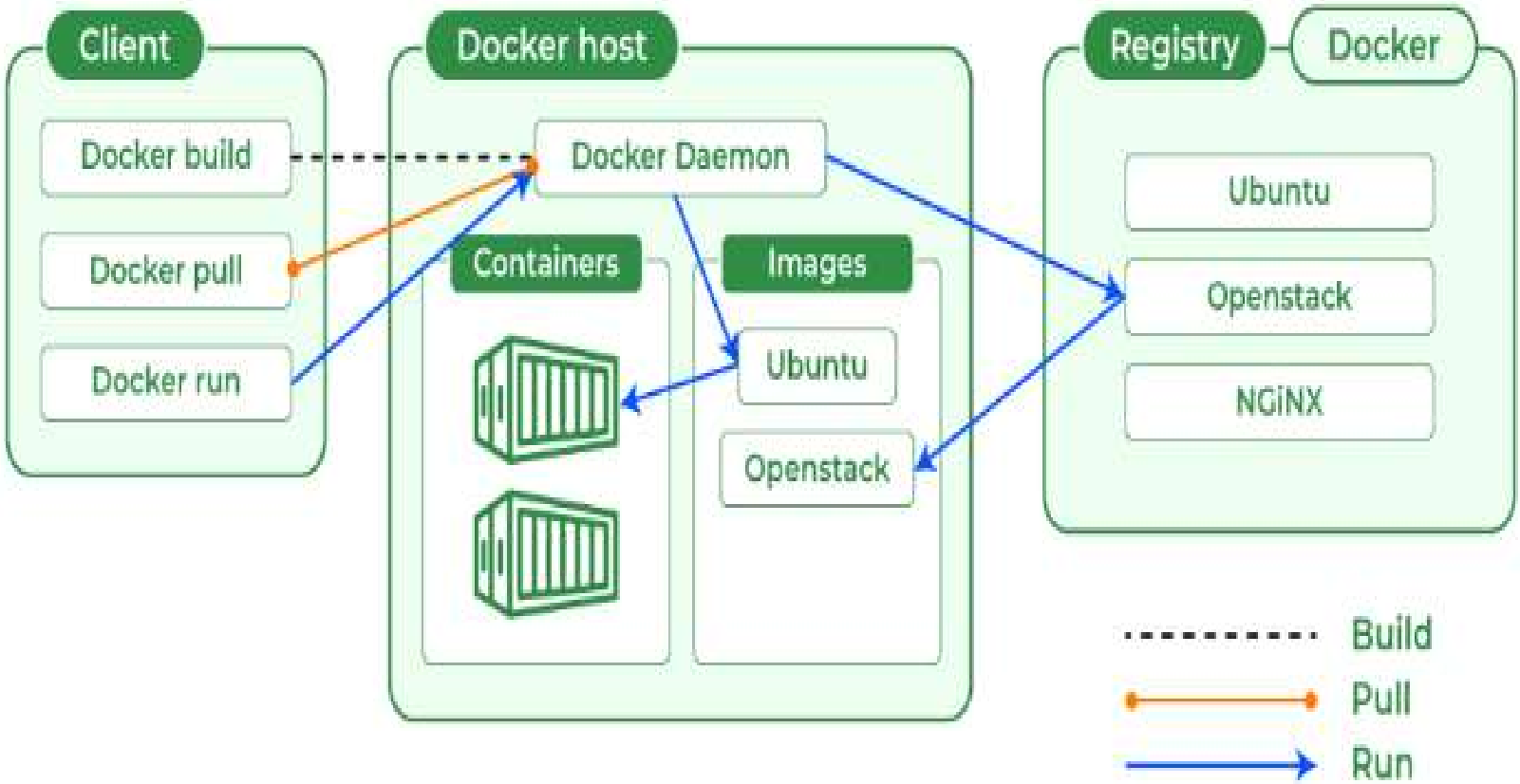
What is Docker Architecture and How Docker Works?

- Docker makes use of **a client-server architecture**. The Docker client talks with the docker daemon which helps in building, running, and distributing the docker containers.
- The Docker client runs with the daemon on the same system or we can connect the Docker client with the Docker daemon remotely. With the help of REST API over a UNIX socket or a network, the docker client and daemon interact with each other.

Docker Engine Architecture

- The software that hosts the containers is named Docker Engine. Docker Engine is a client-server based application. The docker engine has 3 main components:
- **Server:** It is responsible for creating and managing Docker images, containers, networks, and volumes on the Docker. It is referred to as a daemon process.
- **REST API :** It specifies how the applications can interact with the Server and instructs it what to do.
- **Client:** The Client is a docker command-line interface (CLI), that allows us to interact with Docker using the docker commands.

Docker Engine Architecture



Docker Commands

- Through introducing the essential docker commands, docker became a powerful software in streamlining the container management process. It helps in ensuring a seamless development and deployment workflows. The following are the some of docker commands that are used commonly:
- **Docker Run:** It used for launching the containers from images, with specifying the runtime options and commands.
- **Docker Pull:** It fetches the container images from the container registry like Docker Hub to the local machine.
- **Docker ps** : It helps in displaying the running containers along with their important information like container ID, image used and status.
- **Docker Stop** : It helps in halting the running containers gracefully shutting down the processes within them.
- **Docker Start:** It helps in restarting the stopped containers, resuming their operations from the previous state.
- **Docker Login:** It helps to login in to the docker registry enabling the access to private repositories.

Introduction to Kubernetes (K8S)

- Kubernetes is an open-source platform that manages Docker containers in the form of a cluster. Along with the automated deployment and scaling of containers, it provides healing by automatically restarting failed containers and rescheduling them when their hosts die. This capability improves the application’s availability.

What is Kubernetes?

- Kubernetes is an open-source system that automates container deployment tasks. It was originally developed at Google but is now maintained as part of the Cloud Native Computing Foundation (CNCF).
- Kubernetes has risen to prominence because it solves many of the challenges around using containers in production. It makes it easy to launch limitless container replicas, distribute them across multiple physical hosts, and set up networking so users can reach your service.

What is Kubernetes used for?

- Kubernetes is used to manage and scale applications running in containers, which are small, isolated environments. It simplifies the reliable management of numerous apps and services, even when they are distributed across multiple servers.
- Kubernetes automates things like:
- **Starting new apps** when needed.
- **Restarting apps** if they crash.
- **Spreading out work** so that no one part of the system is overloaded.
- **Scaling up or down** based on demand.

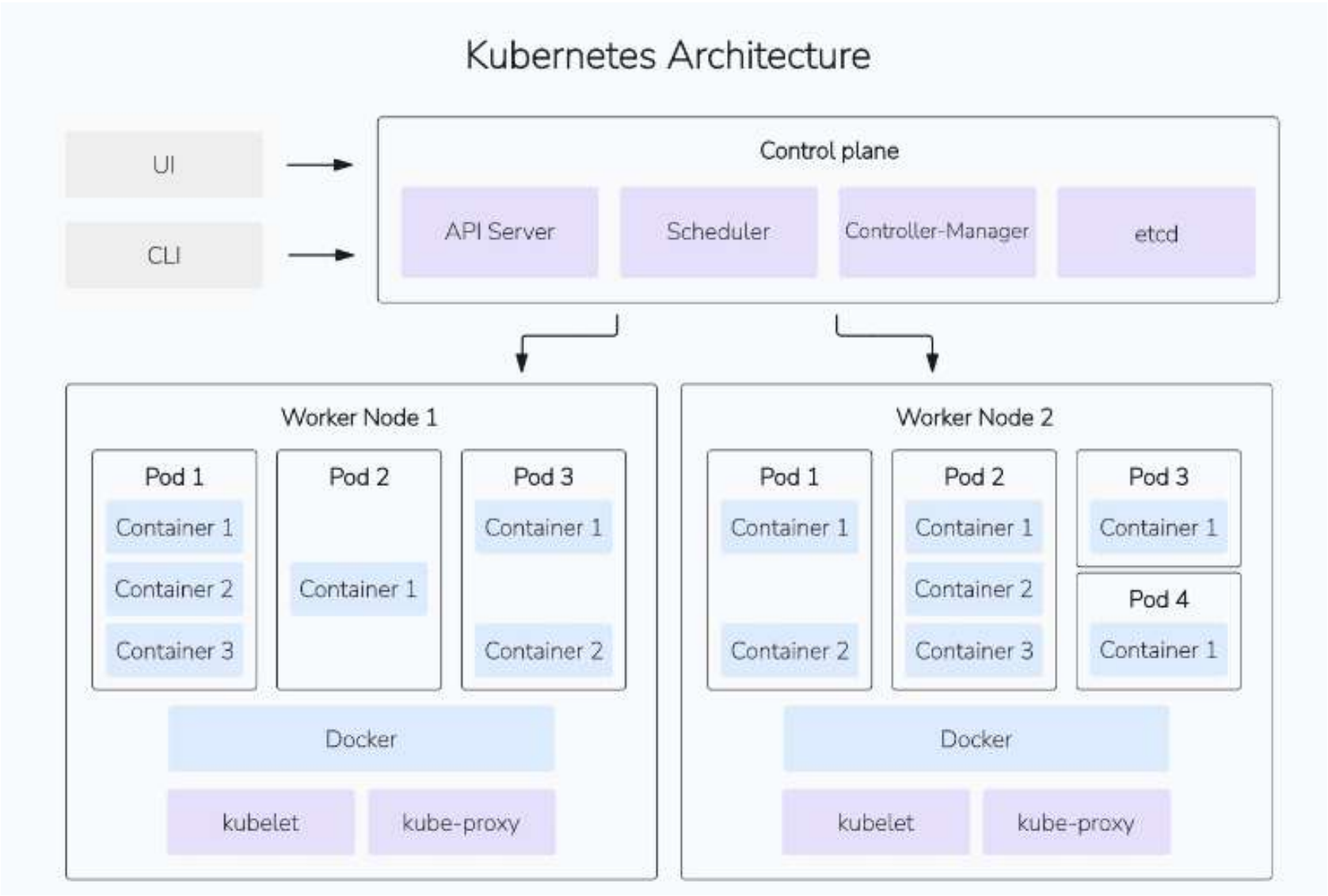
Deploying and Managing Containerized Applications With Kubernetes

- Follow the steps mentioned below to deploy the application in the form of containers.
- **Step 1:** Install kubernetes and setup kubernetes cluster there should be minimum at least one master node and two worker nodes you can set up the kubernetes cluster in any of the cloud which are providing the kubernetes as an service.
- **Step 2:** Know create deployment manifestfile you can create this manifests in the manifest you can specify the exact number of pods are required and what the container image and what types of resources are required after completion of writing the manifestfile apply the file using kubectl command.
- **Step 3:** After creating the pods know you need to expose the service to the outside of the for that you need to write one more manifestfile which contains service type (e.g., LoadBalancer or ClusterIP), ports, and selectors.

Architecture of Kubernetes

- Kubernetes follows the client-server architecture where we have the master installed on one machine and the node on separate Linux machines. It follows the master-slave model, which uses a master to manage Docker containers across multiple Kubernetes nodes.
- A master and its controlled nodes(worker nodes) constitute a **“Kubernetes cluster”**. A developer can deploy an application in the docker containers with the assistance of the Kubernetes master.

Architecture of Kubernetes



Kubernetes elements

- Here are the most important Kubernetes elements:
- **Kubernetes componentDescription**
- **kube-apiserver**This is the part of the control plane that runs the API server. It's the only way to interact with a running Kubernetes cluster. You can issue commands to the API server using the Kubectl CLI or an HTTP client.
- **kube-controller-manager**The controller manager starts and runs Kubernetes' built-in controllers. A controller is essentially an event loop that applies actions after changes in your cluster. They create, scale, and delete objects in response to events such as an API request or increased load.
- **kube-scheduler** The scheduler assigns new Pods (containers) onto the nodes in your cluster. It establishes which nodes can fulfill the Pod's requirements, then selects the most optimal placement to maximize performance and reliability.
- **kubelet**Kubelet is a worker process that runs on each of your nodes. It maintains communication with the Kubernetes control plane to receive its instructions. Kubelet is responsible for pulling container images and starting containers in response to scheduling requests.
- **kube-proxy**Proxy is another component found on individual nodes. It configures the host's networking system so traffic can reach the services in your cluster.
- Kubectl is usually the final piece in a functioning Kubernetes environment. You'll need this CLI to interact with your cluster and its objects.